

Project Design Phase
Problem – Solution Fit Template

Date	28 September 2026
Team ID	6ab221341
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

Problem – Solution Fit Template:

The **Client Scripts & UI Policies Canvas** provides a structured approach to controlling client-side form behavior and user interface dynamics in platforms like ServiceNow. It ensures that the user experience is seamlessly aligned with business logic—delivering dynamic field attributes, validation, and real-time responsiveness directly within the browser.

Purpose:

- **Solve Complex UI Logic:** Address dynamic form requirements (e.g., hiding fields, making fields mandatory, or validating inputs) based on specific user roles or form states.
- **Accelerate Adoption & Efficiency:** Streamline user workflows by reducing form clutter and providing immediate, real-time feedback using no-code UI Policies or low-code Client Scripts.
- **Sharpen Data Quality & Validation:** Enforce strict data entry rules at the browser level (e.g., via onSubmit scripts) before records are sent to the server.
- **Minimize Server Load & Latency:** Use asynchronous calls (GlideAjax) and lightweight client-side execution to handle user interactions without freezing the interface or overburdening the server.
- **Optimize User Context:** Dynamically adapt the interface based on who is logged in (g_user) and what data is being modified (g_form), creating a tailored experience for every user group.

Template:

IMPLEMENTATION CANVAS: CLIENT SCRIPTS & UI POLICIES						
Define CS, fit into CC	1. ON-LOAD & ON-CHANGE TRIGGER CONDITIONS. Specify conditions for execution. E.g., On form load for field X, OR On change of field Y. Consider isLoading & oldValue.	CS	6. REQUIRED API FUNCTIONS (g_form, g_user). Document all required standard API methods. E.g., g_form.setMandatory(), g_form.setReadOnly(), g_form.hasRole(), g_user.getUserID().	CC	5. INTEGRATION & AJAX NEEDS (SERVER-SIDE CALLS). Specify any asynchronous server data retrieval. E.g., Use GlideAjax for complex validation, pre-populating fields based on conditions. List Script Include methods.	AS
	2. CLIENT SCRIPT TYPE & LOGIC. Define the script type (onLoad, onChange, onSubmit, onCellEdit). Write the precise logic and actions for the script, step-by-step. E.g., onFieldX change, check conditions, then set values.	J&P	9. DOM MANIPULATION RISKS & ALTERNATIVES. Identify potential issues. E.g., Minimize browser dependencies. Explore alternative implementations if pure client-side logic is complex. document non-standard DOM usage (as a last resort).	RC	7. UI POLICY ACTION MAPPINGS. List all no-code field actions. E.g., State: Resolved -> field_1 Read-Only, field_2 Mandatory, field_3 Not Visible. Map specific condition groups to field states.	BE
Focus on J&P, tap into BE, understand RC	3. UI POLICY SCRIPTING CONDITIONS For advanced UI Policies (Execute if True/False), define the conditional logic if actions alone are insufficient. E.g., Check g_user roles AND specific field values.	TR	10. DATA VALIDATION & ERROR HANDLING Describe validation checks (especially in onSubmit). Define clear error messages and how to handle incorrect data (g_form.showFieldMsg, g_form.addErrorMessage).	SL	8. DEBUGGING & UNIT TESTING Specify debugging steps (jslog, try/catch, console logging). Document unit test cases (e.g., test onLoad visibility, test onChange value change, test onSubmit rejection).	CH
	4. BEST PRACTICES & OPTIMIZATION Document best practice implementations. E.g., Use 'return false' for onSubmit validation failures, optimize performance, comment code clearly.	EM				
Identify strong TR & EM	UI Policy Actions Mapping					Extract online & offline CH of BE

References:

1. <https://www.ideahackers.network/problem-solution-fit-canvas/>
2. <https://medium.com/@epicantus/problem-solution-fit-canvas-aa3dd59cb4fe>
3. <https://www.servicenow.com/community/developer-articles/client-script/ta-p/2827916>